

Decode Ways Approach Code

91. Decode Ways — Approach

This is a **Dynamic Programming** problem.

The main idea is:

At every position, you have **two possible choices**:

1. Take **one digit**
2. Take **two digits**

But the chosen number must be valid:

- "1" to "9" → valid as a single character
- "10" to "26" → valid as two characters
- "0" alone → invalid
- "06" → invalid because of the leading 0

1. Understand with "226"

We can split it in these valid ways:

```
2 | 2 | 6    -> B B F
2 | 26     -> B Z
22 | 6      -> V F
```

So answer = 3.

At each index, ask:

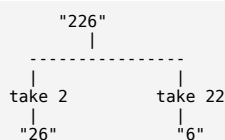
How many ways can I decode the remaining string starting from here?

2. Recursive Thinking

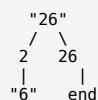
Suppose:

```
s = "226"
```

Start at index 0.



From "26":



So the recurrence is:

```
ways(i) = ways(i + 1) + ways(i + 2)
```

But only when those choices are valid.

3. Important Conditions

Case 1: Current digit is '0'

```
s[i] == '0'
```

Then:

```
return 0
```

Because:

```
0 -> invalid
06 -> invalid
```

Case 2: Single digit

If:

```
s[i] != '0'
```

We can always take that digit.

```
ways(i + 1)
```

Case 3: Two digits

Take:

```
C++
s[i] and s[i + 1]
```

Only if the number is between:

```
10 and 26
```

Then add:

```
ways(i + 2)
```

4. Recursion Tree for "226"

```
helper(0)
Remaining: "226"

      helper(0)
      "226"
     /   \
  take 2  take 22
   /      \
helper(1)  helper(2)
"26"      "6"
 /  \    /
take 2 take 26 take 6
 /      \    |
helper(2) helper(3) helper(3)
"6"      ""      ""
 |
take 6
 |
helper(3)
""

helper(3) = 1
```

Now calculate:

```
helper(2) = 1

helper(1)
= helper(2) + helper(3)
= 1 + 1
= 2

helper(0)
= helper(1) + helper(2)
= 2 + 1
= 3
```

Answer:

```
3
```

5. Memoization Solution

We use:

```
C++
dp[i]
```

where:

$dp[i]$ = number of ways to decode the string starting from index i .

C++ Code

```
C++
class Solution {
public:
    int solve(int i, string& s, vector<int>& dp) {
        // Successfully decoded the complete string
        if (i == s.size()) {
            return 1;
        }

        // '0' cannot be decoded alone
        if (s[i] == '0') {
            return 0;
        }

        // Already calculated
        if (dp[i] != -1) {
            return dp[i];
        }

        // Take one digit
        int ways = solve(i + 1, s, dp);

        // Take two digits if valid
        if (i + 1 < s.size()) {
            int num = (s[i] - '0') * 10 +
                      (s[i + 1] - '0');

            if (num >= 10 && num <= 26) {
                ways += solve(i + 2, s, dp);
            }
        }

        return dp[i] = ways;
    }

    int numDecodings(string s) {
        int n = s.size();
        vector<int> dp(n, -1);
        return solve(0, s, dp);
    }
};
```

6. How the Code Works

For every index i :

```
C++
int ways = solve(i + 1, s, dp);
```

This means:

Take the current digit alone.

For example:

```
"226"
Take "2"
Remaining = "26"
```

Then check whether two digits can be taken:

```
C++
if (num >= 10 && num <= 26) {
    ways += solve(i + 2, s, dp);
}
```

For:

```
"226"
```

At index 0:

```
2  -> valid
22 -> valid
```

Therefore:

```
ways = solve(1) + solve(2)
```

7. Tabulation Approach

The memoization state is:

```
dp[i] = number of ways to decode from index i to end
```

Since `dp[i]` depends on:

```
dp[i + 1]
dp[i + 2]
```

we fill the DP array **from right to left**.

Base Case

Imagine:

```
s = "226"
```

After the last character:

```
index:  0   1   2   3
        2   2   6  END
```

At:

```
dp[3] = 1
```

Why?

Because if we have successfully reached the end, that represents **one valid way**.

Tabulation Code

```
C++
class Solution {
public:
    int numDecodings(string s) {
        int n = s.size();
        vector<int> dp(n + 1, 0);

        // Base case
        dp[n] = 1;

        // Fill from right to left
        for (int i = n - 1; i >= 0; i--) {
            // '0' cannot be decoded alone
            if (s[i] == '0') {
                dp[i] = 0;
                continue;
            }

            // Take one digit
            dp[i] = dp[i + 1];

            // Take two digits
            if (i + 1 < n) {
                int num = (s[i] - '0') * 10 +
                           (s[i + 1] - '0');

                if (num >= 10 && num <= 26) {
                    dp[i] += dp[i + 2];
                }
            }
        }

        return dp[0];
    }
}
```

```
};
```

8. Dry Run: "226"

Initial:

```
dp = [0, 0, 0, 1]
      ↑
      dp[3]
```

Because:

```
dp[3] = 1
```

i = 2

Character:

```
6
```

Single digit is valid:

```
dp[2] = dp[3]
      = 1
```

```
dp = [0, 0, 1, 1]
```

i = 1

Remaining:

```
"26"
```

Take "2":

```
dp[1] = dp[2]
      = 1
```

Take "26":

```
26 is valid
```

So:

```
dp[1] += dp[3]
dp[1] = 1 + 1
      = 2
```

```
dp = [0, 2, 1, 1]
```

i = 0

Remaining:

```
"226"
```

Take "2":

```
dp[0] = dp[1]
      = 2
```

Take "22":

```
22 is valid
```

Therefore:

```
dp[0] += dp[2]
dp[0] = 2 + 1
      = 3
```

Final:

```
dp = [3, 2, 1, 1]
```

Answer:

3

9. Most Important Edge Cases

"0"

0 cannot be decoded

Answer = 0

"06"

06 is invalid

Answer = 0

"10"

1 | 0 
10 

Answer = 1

"20"

2 | 0 
20 

Answer = 1

"27"

2 | 7 
27  because > 26

Answer = 1

"100"

10 | 0 

Answer = 0

Complexity

Memoization

Time: $O(n)$
Space: $O(n)$

Tabulation

Time: $O(n)$
Space: $O(n)$

The core pattern to remember is:

$dp[i] = dp[i + 1] + dp[i + 2]$

But **only add each part when the corresponding 1-digit or 2-digit decoding is valid**. This is essentially a **1D DP / Fibonacci-style pattern with validity conditions**.

ChatGPT can make mistakes. Check important info.